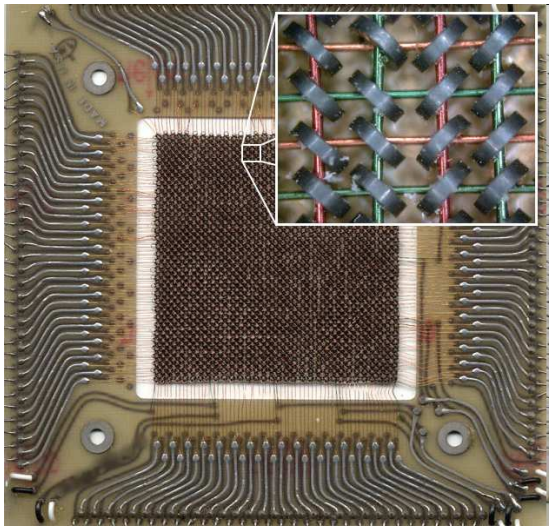
University of British Columbia
Electrical and Computer Engineering
Introduction to Microcomputers EECE259

Lecture 16: Memory in the 8051 Microcontroller

Dr. Jesús Calviño-Fraga P.Eng.
Department of Electrical and Computer Engineering, UBC
Office: KAIS 3024
E-mail: jesusc@ece.ubc.ca
Phone: (604)-827-5387

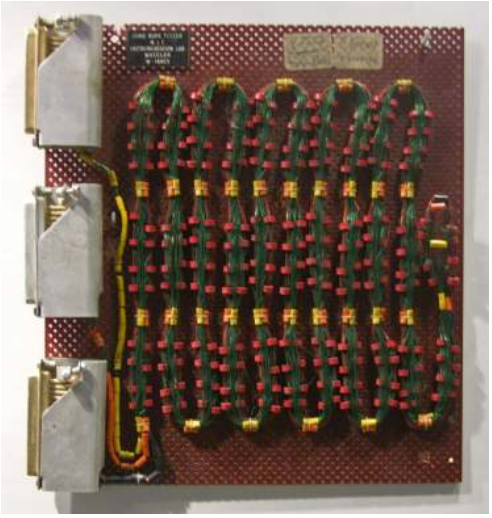
June 10, 2014

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.



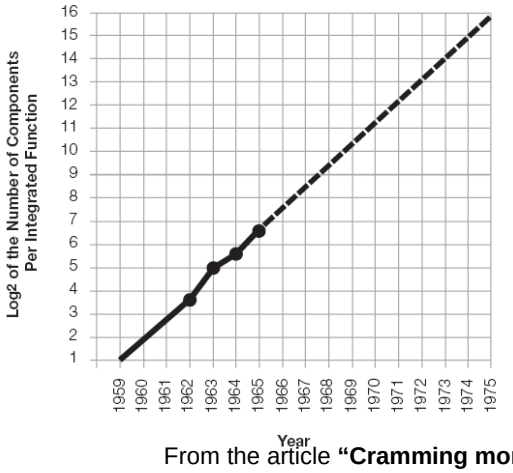
Magnetic-core memory, used as RAM

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.



Core rope memory, used as ROM

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.



In 2005 Intel offered 10000$ for an original Electronics Magazine where the article appeared!

From the article **"Cramming more components onto integrated circuits"** By Gordon E. Moore published in *Electronics Magazine* Volume 38, Number 8, April 19, 1965

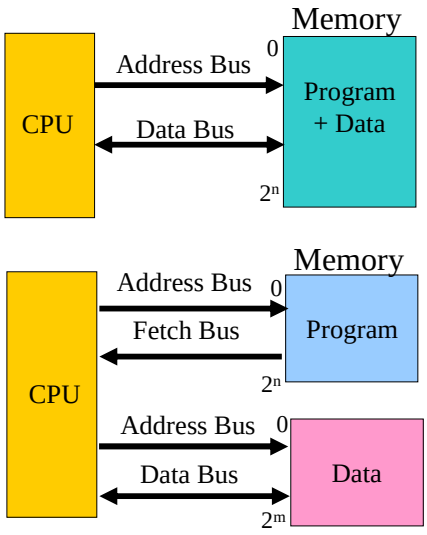
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Objectives

- Understand the differences and uses of bit, data, idata, xdata, and code memory in the 8051 microcontroller.
- Use the appropriate assembly instruction for each type of memory.
- Define variables using assembly code in the bit, data, idata, xdata, and code memory spaces.
- Use code memory to create look-up tables.
- Understand memory device access timing.
- Understand the multiplexed address/data bus in the 8051.
- Design memory decoding circuitry.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Microcontroller Architectures

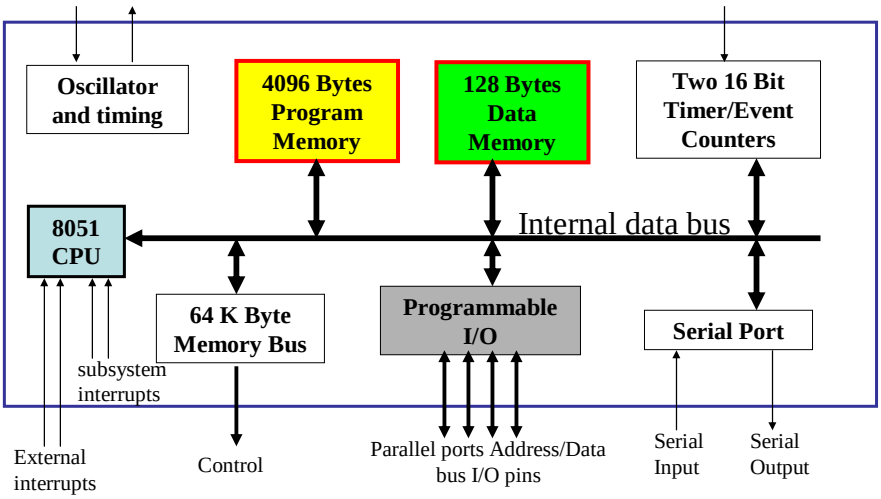


Von Neumann Architecture

Harvard Architecture

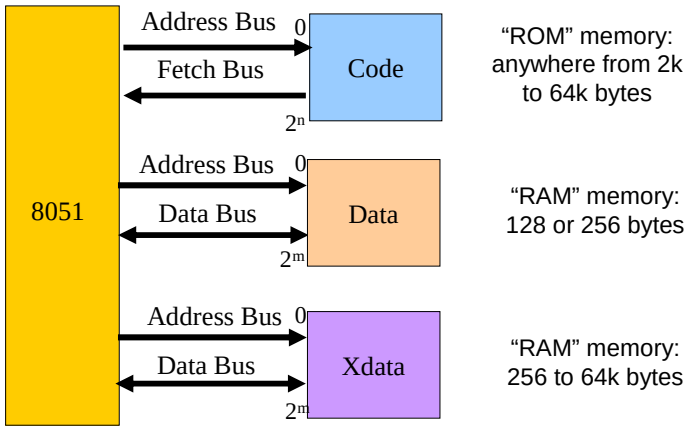
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

“Original” 8051 Microcontroller



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

“Modern” 8051 Microcontroller Memory



“ROM” memory: anywhere from 2k to 64k bytes

“RAM” memory: 128 or 256 bytes

“RAM” memory: 256 to 64k bytes

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

ROM Types

- ROM (Read Only Memory): can only be read by the microcontroller.
  - PROM: Non erasable.
  - EPROM: Erasable using ultraviolet light.
  - EEPROM: Erasable using electricity! Can be rewritten in a byte by byte basis. Good for non-volatile storage.
  - Flash: Erasable. Normally you'll need to erase a significant part of the memory before write anything to it. Cheap, inexpensive, fast, and reliable. DO NOT BUY MICROCONTROLLERS WITHOUT FLASH MEMORY.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

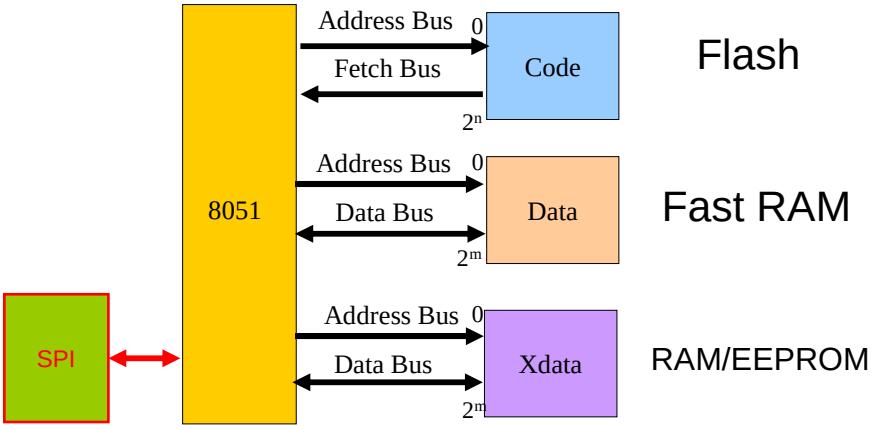
RAM Types

RAM (Random Access Memory):

- Dynamic: Ultra dense, but need to be refreshed periodically or the content will fade away. One memory cell is basically a capacitor!
- Static: Not as dense, but simpler to use. One memory cell is basically a flip-flop.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

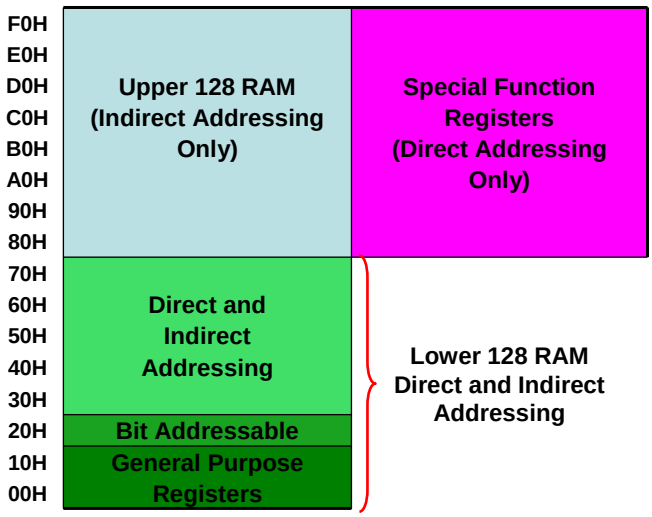
“Modern” 8051 Microcontroller Memory



Flash: storage of Megabytes!

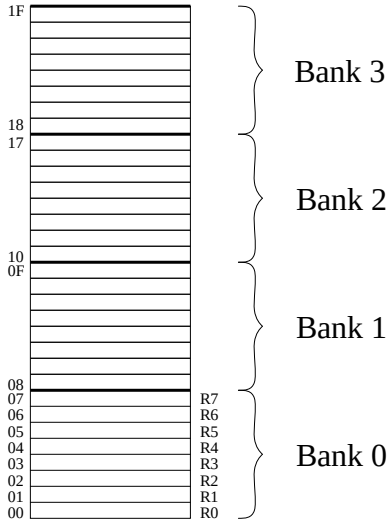
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

On-Chip DATA Memory: RAM



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

General Purpose Registers



- **There are efficient MOV instructions that target the registers directly:**

  MOV R7, #55H ; takes 2 bytes

  MOV 07H, #55H ; takes 3 bytes

- **To change the register bank in use set/reset the RS0 and RS1 bits in PSW:**

RS1	RS0	Selects
0	0	Bank 0
0	1	Bank 1
1	0	Bank 2
1	1	Bank 3

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Bit Addressable Memory

2F	7F	7E	7D	7C	7B	7A	79	78
2E	77	76	75	74	73	72	71	70
2D	6F	6E	6D	6C	6B	6A	69	68
2C	67	66	65	64	63	62	61	60
2B	5F	5E	5D	5C	5B	5A	59	58
2A	57	56	55	54	53	52	51	50
29	4F	4E	4D	4C	4B	4A	49	48
28	47	46	45	44	43	42	41	40
27	3F	3E	3D	3C	3B	3A	39	38
26	37	36	35	34	33	32	31	30
25	2F	2E	2D	2C	2B	2A	29	28
24	27	26	25	24	23	22	21	20
23	1F	1E	1D	1C	1B	1A	19	18
22	17	16	15	14	13	12	11	10
21	0F	0E	0D	0C	0B	0A	09	08
20	07	06	05	04	03	02	01	00

20h – 2Fh (16 locations X 8-bits = 128 bits)

Bit addressing:

setb 1Ah ; Two bytes

Byte addressing:

orl 23h, #04H ; Three bytes

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Direct addressable memory

- If address < 80H: the target is general purpose RAM.
- If address ≥ 80H: the target is Special Function Registers

```
mov 30H, #0AAH ; Move number 0AAH to RAM 30H
mov 80H, #04H  ; Move number 04H to SFR 80H (P0)
mov P0, #04H   ; Move number 04H to SFR 80H (P0)
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Bit Addressable SFRs

- If the address of a SFR ends in “0” or “8”, the bits of the SFR are bit addressable.

```
orl 80H, #04H ; Set bit 2 of SFR 80H (P0)
orl P0, #04H  ; Set bit 2 of SFR 80H (P0)
setb P0.2     ; Set bit 2 of SFR 80H (P0)
setb 82H      ; Set bit 2 of SFR 80H (P0)
orl DPH, #100 ; DPH is at address 83H, not bit addressable
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Indirectly addressable memory

- All the data memory in the 8051 is indirectly addressable.
- The SFRs are not indirectly addressable.
- To indirectly access memory we use the indexing registers R0 and R1.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Indirectly addressable memory

Write a program that clears (set to zero) 32 consecutive bytes from internal RAM starting at address 70H.

```
    mov R0, #70H ; Starting address we want to clear
    mov R7, #32  ; How many bytes we want to clear
L1:
    mov @R0, #0  ; Set memory pointed by R0 to zero
    inc R0       ; Point to the next byte
    djnz R7, L1  ; Repeat as many times as needed
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Expanded RAM

- In the original 8051 expanded (or external) RAM was only available as additional memory chips. Up to 64k!
- Newer 8051's may include some built in expanded RAM.
- Expanded RAM can only be accessed indirectly using the MOVX instruction.
- The MOVX instruction uses either the DPTR (DPL, DPH), R0, or R1 as indexes.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Expanded RAM Access

Write a program that sets 50H bytes from expanded memory at address 80H to 255

```
    mov A, #0FFH        ; Value we want to set the memory to
    mov DPTR, #128      ; Starting address we want to set
    mov R0, #50H        ; How many bytes we want to set
X1:
    movx @DPTR, A       ; Set memory pointed by R0 to zero
    inc DPTR            ; Point to the next memory location
    djnz R0, X1         ; Repeat as many times as needed
```

Some 8051's derivatives have two DPTR registers to simplify memory access. Check the data sheet for 'dual data pointers'.

Remember: 80H=128, 0FFH=255

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Code Memory

- Code memory can only be accessed indirectly using the MOVC instruction.
- The MOVC instruction uses either the DPTR (DPL, DPH), or the accumulator as indexes.
- Very useful when working with look-up tables!
- We can only READ from code memory.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Code Memory Access

Write a program to copy 8 bytes from code memory at address 1000H to data memory at address A0H

```
; Copy from code memory to data memory
    mov DPTR, #1000H    ; Source address in code memory
    mov R0, #A0H        ; Destination address in data memory
    mov R2, #08H        ; # of bytes we want to copy
C1:
    clr A
    movc A, @A+DPTR     ; Read from code memory
    mov @R0, A          ; Copy to data memory!
    inc DPTR            ; Point to next byte in code memory
    inc R0              ; Point to next byte in data memory
    djnz R2, C1         ; Repeat as many times as needed
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Defining Variables in Assembly Source Code

- Use the segment directives (BSEG, DSEG, ISEG, XSEG, CSEG) to select bit, data, idata, xdata, or code memory.
- Use the origin directive override (AT) to set the beginning of the segment.
- USE the space allocation directives (DS, DBIT, DB, DW) to define the variables or their values.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Defining Variables in Assembly Source Code

```
; Defining bit variables
BSEG

ALARM: dbit 1
ONOFF: dbit 1

; Defining variables in data memory
DSEG at 30H

COUNT_HIGH: ds 1
COUNT_LOW:  ds 1
COUNT:      ds 2
BUFFER:     ds 16
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Defining Variables in Assembly Source Code

```
; Defining xdata variables
XSEG at 80H

Sum:    ds 2
Adjust: ds 20

; Defining constants in code memory
CSEG at 1000H

Message: db 'Hello there…'
         db 0ah, 0dh ; ASCII code for nl/cr
         db 0
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Lookup tables: some references

- Art of Assembly: http://webster.cs.ucr.edu/AoA/DOS/ch09/CH09-7.html
- Google: “Wikipedia lookup table”
- Google: “8051 lookup table”
- Check page 115 of textbook: “Indexable Data Structures”

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example: 8-bit ADC value to voltage

- Assume the maximum voltage the ADC can read is 3.3V. Therefore the least significant bit represents $3.3V/2^8=0.0129V$
- Lookup table:

```
mytable:
    dw 00000 ; 00H
    dw 00129 ; 01H
    dw 00258 ; 02H
    .
    .
    dw 32871 ; FFH
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Creating the lookup table…

Write a program in C, Matlab, Java, Excel, etc. to create the assembly table for you:

```
#include <stdio.h>
#include <stdlib.h>

void main (void)
{
   int j;
   double k=3.3/256.0;

   printf("mytable:\n");
   for(j=0; j<256; j++)
          printf("   dw %05d ; %02xH\n", (int)(j*k*10000), j);
}
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

8-bit ADC value to voltage

```
mov dptr, #mytable
mov a, R7 ; R7 has the conversion value
mov b, #2
mul ab
add a, dpl
mov dpl, a
mov a, b
addc a, dph
mov dph, a
```

dptr=mytable+(2*R7);

```
clr a
movc a, @a+dptr
mov R0, a ; High part from the lookup table
inc dptr
clr a
movc a, @a+dptr
mov R1, a ; Low part from the lookup table
```

R0=mytable[2*R7];
R1=mytable[2*R7+1];

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

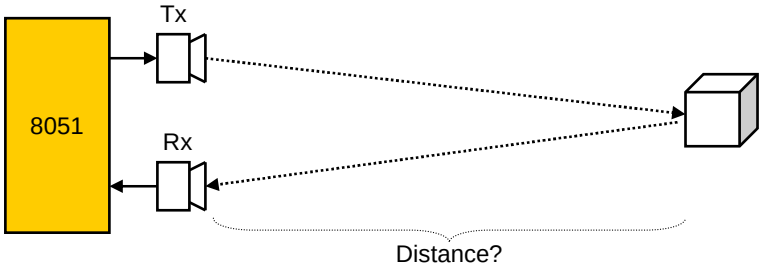
Look-up by Index or Value?

- Sometimes we know the index in the table, and what we need is the value. As in the previous example.
- Sometimes what we know is the (approximate) value in the table, and what we are looking for is the index!
- To find the index, we can scan the whole table and compare the known value until we get a match! A more efficient 'binary search algorithm' can be implemented instead (see textbook, example 3.3).

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Table look-up by value

- Consider measuring distance using ultrasound:



For a microcontroller is fairly easy to measure the time a burst of sound takes to travel from the transmitter back to the receiver. With that and the speed of sound we can calculate the distance:

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Table look-up by value

$$v = \frac{d}{t} = \frac{34029}{2}\frac{cm}{s} = 17015\frac{cm}{s}$$

With this equation, we create a look-up table

The speed of sound is 340.29 m/s, but since in our case the wave has to travel twice the distance, the speed of sound we use is half of that.

d (cm)	t (µs)
1	58
2	118
3	176
4	235
5	294
6	353
7	411
.	.
.	.
.	.
250	14693

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Table look-up by value

```
;Find the distance from
;the table of times
Find_Distance:
    mov R1, #0
    mov dptr, #Time2Dist
next_d:
    clr a
    movc a, @a+dptr
    mov R6, a
    clr a
    inc dptr
    movc a, @a+dptr
    mov R7, a
```

```
;(R6,R7)-MyTime
    clr c
    mov a, R7
    subb a, MyTime+1
    mov a, R6
    subb a, MyTime
    jnc done
    inc R1
    inc dptr
    cjne R1, #251, next_d
done:
    ;R1 has the distance!
    ret
```

```
Time2Dist:
  DW    58
  DW    118
  DW    176
  DW    235
  DW    294
  DW    353
  DW    411
  .
  .
  .
  DW    14693
```

Works, but very inefficient. This can be done faster using a binary search. See exercises at the end.

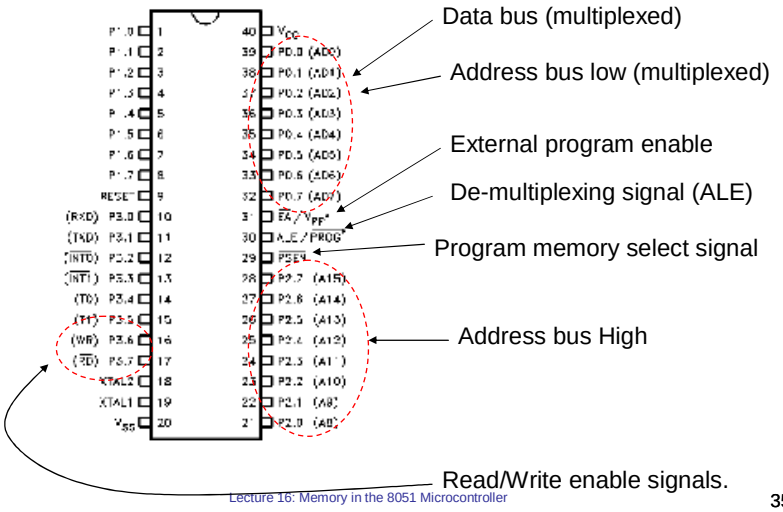
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

8051 External/Expanded Memory

- Original 8051:
  - 4k of internal code memory (Factory masked ROM)
  - Only 128 bytes of RAM.
- Original 8751:
  - 4k of internal code memory (EPROM)
  - Again only 128 bytes of RAM.
- But allowed us to add up to:
  - 64k of code memory (FLASH, EPROM).
  - 64k of RAM.
  - Extra I/O ports or devices mapped as RAM.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

How?



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

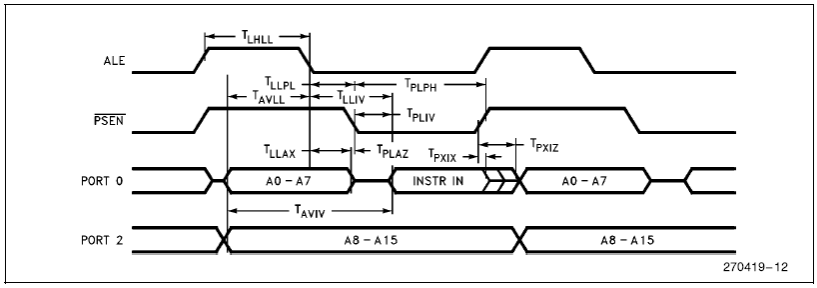
Warning!

- If you add external memory to an standard 8051 microcontroller both P0 and P2 are used to access the memory ONLY and can not be used for anything else!
- Also, pins P3.6 (WR/) and P3.7 (RD/) are used to access the memory and can not be used for anything else!
- This doesn't apply to the DE2-8052 where the memory bus is completely separated from the port bus!
- This doesn't apply to many modern 8051s where you can not access additional external memory at all!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Program Memory Read Cycle

EXTERNAL PROGRAM MEMORY READ CYCLE

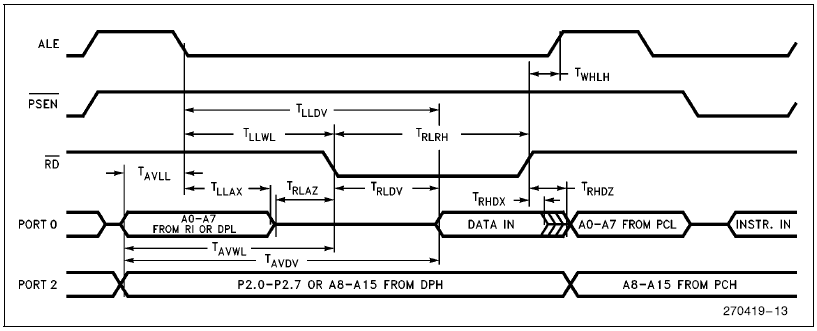


Taken from "MCS-51 CHMOS SINGLE-CHIP 8-BIT MICROCONTROLLER" datasheet © Intel

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Data Memory Read Cycle

EXTERNAL DATA MEMORY READ CYCLE

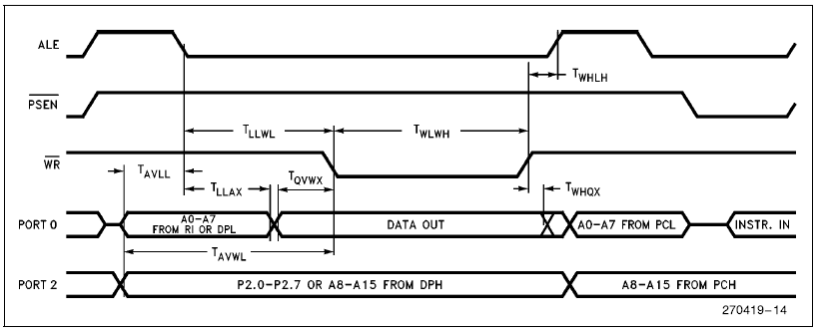


Taken from "MCS-51 CHMOS SINGLE-CHIP 8-BIT MICROCONTROLLER" datasheet © Intel

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Data Memory Write Cycle

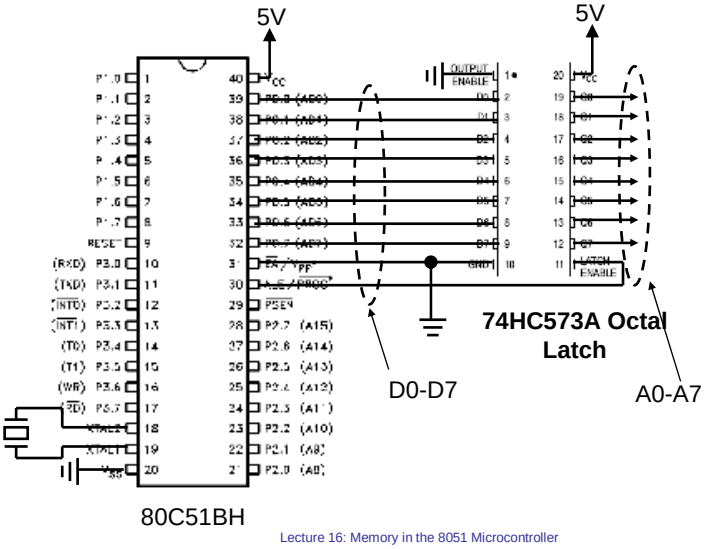
EXTERNAL DATA MEMORY WRITE CYCLE



Taken from "MCS-51 CHMOS SINGLE-CHIP 8-BIT MICROCONTROLLER" datasheet © Intel

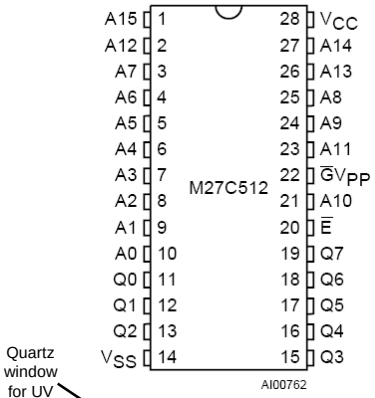
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

First task, separate D0-D7 from A0-A7

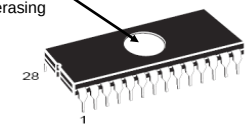


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Adding External Code Memory: 512 Kbit (64K x 8) UV EPROM



Quartz window for UV erasing



27C512 Features

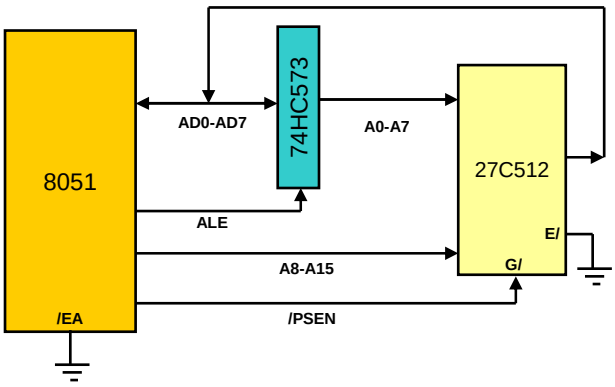
- 5V ± 10% supply voltage in read operation
- Access time: 45 ns
- Low power "CMOS" consumption:
  - Active current 30 mA
  - Standby current 100 µA
- Programming voltage: 12.75 V ± 0.25 V
- Programming time around 6 s.
- Electronic Signature
  - Manufacturer code: 20h
  - Device code: 3Dh

6.85$ each!!!

I am surprised this chip is still available! I first used this chip in the late 1980's

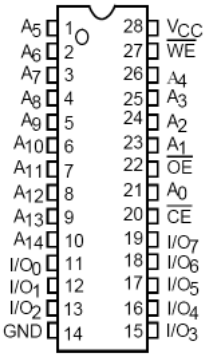
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Adding External Code Memory: 512 Kbit (64K x 8) UV EPROM



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Adding External RAM Memory: 256K (32K x 8) Static RAM



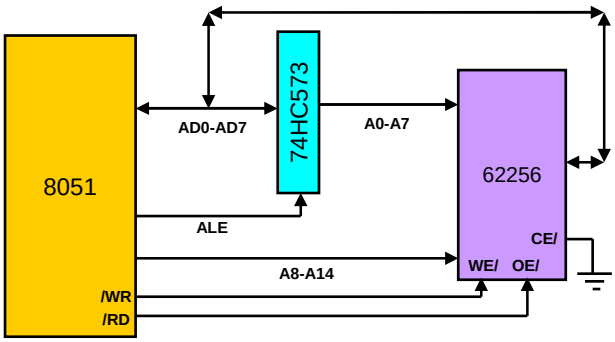
CY62256N

Features
- **High speed: 55 ns**
- **Voltage range: 4.5V–5.5V operation**
- **Low active power**
  — 275 mW (max.)
- **Low standby power (LL version)**
  — 82.5 µW (max.)
- **Easy memory expansion with CE and OE features**
- **TTL-compatible inputs and outputs**
- **Automatic power-down when deselected**
- **CMOS for optimum speed/power**
- **Available in pb-free and non Pb-free 28-lead (600-mil) PDIP, 28-lead (300-mil) narrow SOIC, 28-lead TSOP-I and 28-lead Reverse TSOP-I packages**

3.49$ each!!!

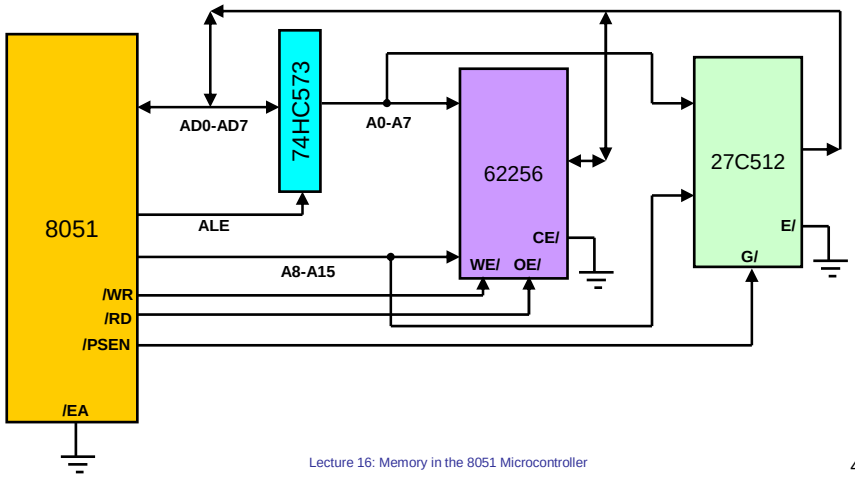
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Adding External RAM Memory: 256K (32K x 8) Static RAM



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Adding External Code &Memory External RAM



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Memory Mapping

- For the 8051, normally done with the expanded memory RAM space only.
- We can map memory or I/O devices.
- For example, we can add to our system:
  - Dual port RAM.
  - Flash (read/write).
  - EEPROM.
  - Paged memory: more than 64k of ram in m pages of nk. This is also called memory banking.
- Many of these high density for-storage-only memories can be better handled using SPI ICs (SPI: Serial Peripheral Interface) or SD card memories.

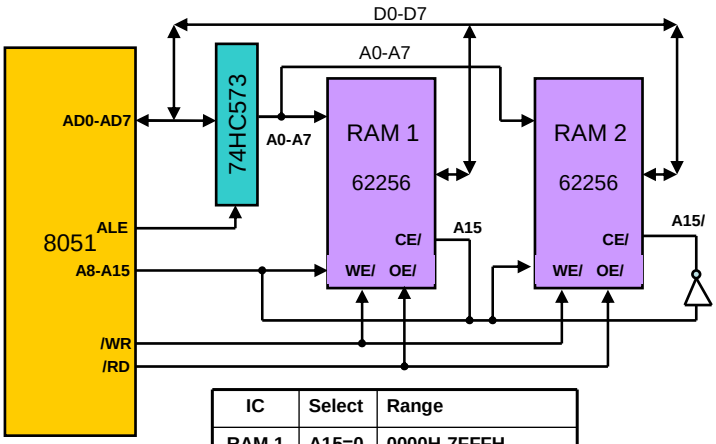
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Memory Mapping

- Decoding:
  - Take A0 to A15, RD/, WR/, PSEN/, and generate the chip enable for the different devices.
- We need to use logic gates to create the decoded functions:
  - Small CPLD or even FPGA!
  - Discrete logic gates.
  - Specific purpose logic devices, for example decoders such as the 74HC138 and 74HC139.
  - Some microprocessors/microcontrollers include configurable memory selection pins!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: Add two x 32k RAM



IC	Select	Range
RAM 1	A15=0	0000H-7FFFH
RAM 2	A15=1	8000H-FFFFH

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 2

Design a chip select generator for the 8051 microcontroller to access the following memories and peripherals as expanded memory. All the chip enable signals are active low.

IC	Name	Range	Address bits used
RAM	CS0/	0000H-3FFFH	A15=0, A14=0
EEPROM	CS1/	4000H-7FFFH	A15=0, A14=1
SWITCHES	CS2/	8000H-BF00H	A15=1, A14=0
LEDS	CS3/	C000H-FFFFH	A15=1, A14=1

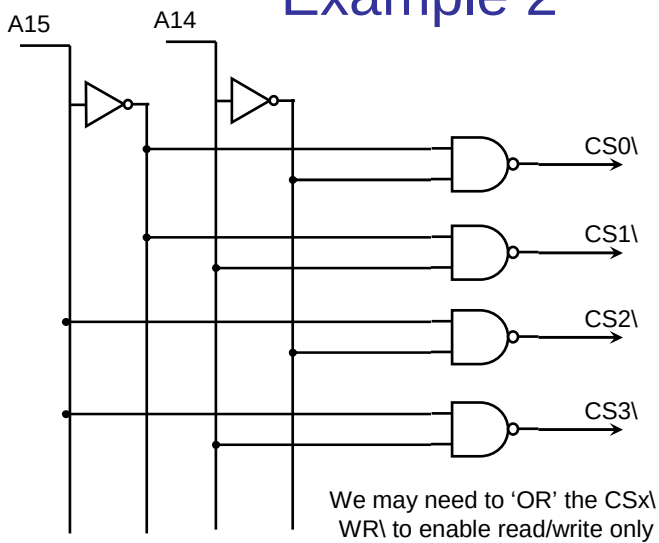
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 2

Inputs		Outputs			
A15	A14	CS0/	CS1/	CS2/	CS3/
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 2



We may need to 'OR' the CSx\ with RD\ or WR\ to enable read/write only operations

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 2

74HC139

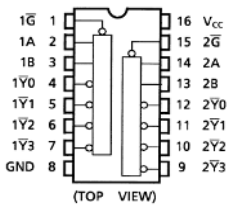


Figure 4.18 in textbook… This figure is frequently used in exams!

Truth Table

Inputs			Outputs				Selected Output
Enable	Select						
$\overline{G}$	B	A	$\overline{Y}0$	$\overline{Y}1$	$\overline{Y}2$	$\overline{Y}3$	
H	X	X	H	H	H	H	None
L	L	L	L	H	H	H	$\overline{Y}0$
L	L	H	H	L	H	H	$\overline{Y}1$
L	H	L	H	H	L	H	$\overline{Y}2$
L	H	H	H	H	H	L	$\overline{Y}3$

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

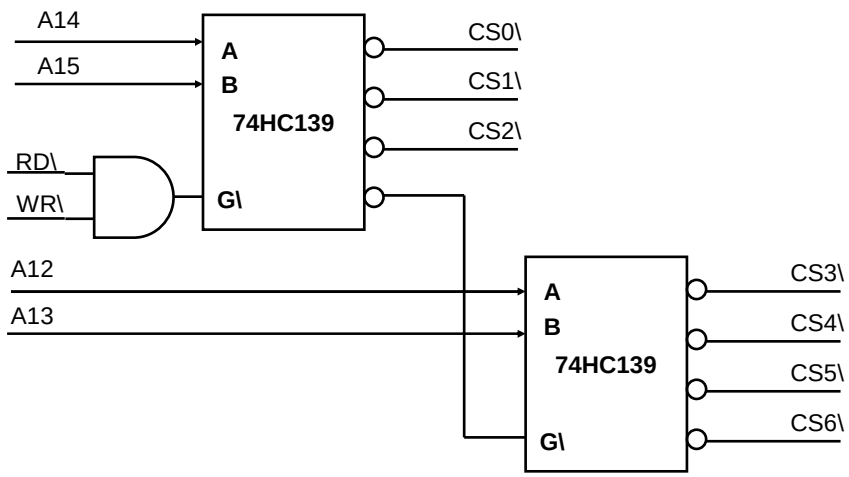
Example 3: Multilevel Decoding

Design a chip select generator for the 8051 microcontroller to access the following memories and peripherals as expanded memory. All the chip enable signals are active low. Use the 74HC139 IC if possible.

IC	Name	Range	Address bits used
RAM	CS0/	0000H-3FFFH	A15=0, A14=0, A13=X, A12=X
EEPROM	CS1/	4000H-7FFFH	A15=0, A14=1, A13=X, A12=X
SWITCHES	CS2/	8000H-BF00H	A15=1, A14=0, A13=X, A12=X
LEDS 0	CS3/	C000H-CFFFH	A15=1, A14=1, A13=0, A12=0
LEDS 1	CS4/	D000H-DFFFH	A15=1, A14=1, A13=0, A12=1
LEDS 2	CS5/	E000H-EFFFH	A15=1, A14=1, A13=1, A12=0
LEDS 3	CS6/	F000H-FFFFH	A15=1, A14=1, A13=1, A12=1

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 3: Multilevel Decoding



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 3: Multilevel Decoding

```
; Somewhere at the top of your code

MY_RAM      XDATA   00000H
MY_EEPROM   XDATA   04000H
MY_SWITCHES XDATA   08000H
LEDS_0      XDATA   0C000H
LEDS_1      XDATA   0D000H
LEDS_2      XDATA   0E000H
LEDS_3      XDATA   0F000H

; Somewhere in your code

mov dptr, #LEDS_0

mov a, #0

movx @dptr, a ; Turn on all LEDs
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Exercises

- The speed of sound for dry air depends on the temperature T_C in ^oC, as given by the formula below. Write a look-up table to find the average temperature of a three meter wide room using the ultrasonic transceiver presented above. Assume the ultrasonic transceiver is attached to one wall, and the sound bounces back from the opposite wall. Generate the table for T_C =0 to 63 ^oC.

$$(331.3 + 0.606 * T_C)\frac{m}{s}$$

- Modify the algorithm used to scan the look-up table "by value" presented in the slides above so it uses a more efficient binary search.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Exercises

•Design an extended 'XRAM' memory decoder for the 8051 microcontroller so that:

Address range	Function	Access type
0000H to 7FFFH	RAM (32k)	Read/Write
8000H to 8FFFH	EEPROM (4k)	Read/Write
9000H to 9FFFH	RAM (4k)	Read/Write
A000H to AFFFH	Input	Read Only
B000H to BFFFH	Output	Write Only
C000H to CFFFH	EPROM	Read Only
D000H to DFFFH	Reserved for future use	Read/Write
E000H to EFFFH	Reserved for future use	Read/Write
F000H to FFFFH	Reserved for future use	Read/Write

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Exercises

- A 1 Megabyte (1048576 bytes; 20 address bits: A0 to A19, 8 data bits: D0 to D7, Wr', Rd', and Cs') Flash Memory needs to be connected to an 8051 microcontroller as expanded external RAM. The lower 32k (A15=0, addresses 0000H to 7FFFH) of the available XRAM memory space in the 8051 is already used for RAM for general purpose variables. Design a circuit to connect the Flash memory to the microcontroller as 32 banks of 32k bytes using ports P1.0 to P1.4 to select each bank. The active Flash bank should be mapped in the upper 32k bytes of the microcontroller memory space: from addresses 8000H to FFFFH.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.